

DMF Benchmarks - user manual

Configuring and running dmf-bench

DMF Benchmarks

August 9, 2026

1. Purpose

This manual explains how to use **dmf-bench**: how to prepare a configuration, validate it, run a benchmark, and read the results.

The document is not tied to a single smoke dataset. Examples use concrete paths so they can be copied, but the flow is the same for LoCoMo, LongMemEval, DMF, and Mem0.

dmf-bench runs native-only benchmarks inside Docker. The old protocol concept no longer exists: every run uses the native Docker pipeline.

2. Components

A standard execution uses these components:

- **benchmark**: the container that runs **dmf-bench**.
- **qdrant**: the local vector database used by the frameworks.
- **artifact-api**: a read-only API for reading runs and artifacts.
- **/bench/config**: read-only mounted configurations.
- **/bench/cache**: writable cache for materialized datasets, models, and temporary data.
- **/bench/runs**: writable volume where results are published.

Provider keys must never be written into JSON files. Pass them through environment variables, for example **OPENAI_API_KEY**.

3. Prerequisites

You need an image or environment where the **dmf-bench** CLI is installed.

```
dmf-bench --help
```

If you use an image published to GHCR:

```
docker login ghcr.io
```

Create a local **.env** file in the repository root:

```
DMF_BENCH_IMAGE=ghcr.io/ORG/IMAGE:TAG
OPENAI_API_KEY=paste_the_key_here
HF_HUB_OFFLINE=1
```

HF_HUB_OFFLINE=1 prevents implicit Hugging Face model downloads during an official run. If the cache does not already contain the required models, the run may fail; prepare the cache before the run.

No Qdrant API key is needed when Qdrant runs locally without authentication.

4. Configuration

An experiment is described by one JSON file with `schema_version: 2`.

The configuration is the scientific contract of a run. It says what benchmark is executed, which framework is evaluated, which dataset is used, which model answers, which model judges, where state is stored, and which reports must be produced.

A config should be treated as immutable once a run is considered official. If you change the dataset, framework config, selected units, model names, model parameters, or evaluation requirements, you are defining a different experiment.

The top-level structure is:

- `schema_version`: must be 2. Older benchmark config formats are rejected.
- `experiment_id`: stable human-readable experiment name. It should describe the benchmark, framework, and scope.
- `benchmark`: task to execute, either `locomo` or `longmemeval`.
- `framework`: memory framework under test, either `dmf` or `mem0`.
- `runtime`: operational paths and process settings. This includes root directory, run directory, cache directory, metrics port, and log level.
- `framework_config`: framework-specific TOML or YAML file plus SHA-256. This pins the DMF/Mem0 runtime configuration used by the experiment.
- `qdrant`: how the benchmark reaches Qdrant. The URL is read from an environment variable, normally `QDRANT_URL`.
- `dataset`: dataset declaration. It can point to an already materialized JSON file or to a registry entry that `dmf-bench` materializes before the run.
- `selection`: exact benchmark units to execute and optional filters. This controls the experiment size and order.
- `models`: provider, requested model, generation parameters, timeout, rate limit, and retry policy for answerer and judge.
- `evaluation`: reports that must or may be produced. Missing required reports make the run fail.
- `artifact_store`: destination for committed artifacts, normally the same directory as `runtime.runs_dir`.

The most important rule is separation of responsibilities:

- JSON config files define reproducible experiment inputs.
- Environment variables provide secrets and deployment endpoints.
- Run artifacts record what actually happened.

Do not put provider API keys in the JSON config. For example, `models.answerer.provider` may be `openai`, but the actual key must come from `OPENAI_API_KEY`. Similarly, `qdrant.endpoint_env` normally contains the string `QDRANT_URL`; the actual URL is read from that environment variable at runtime.

Path rules

When running inside the official Docker image, paths normally live under `/bench`:

<code>/bench/config</code>	read-only configuration files
<code>/bench/cache</code>	writable cache for datasets, models, and temporary data
<code>/bench/runs</code>	writable output directory for committed run artifacts

Paths used by the benchmark must be absolute inside the runtime environment and must stay under `runtime.root`, normally `/bench`. This prevents a config from accidentally reading or writing unrelated host files.

For example:

```
{
  "runtime": {
    "root": "/bench",
```

```

    "runs_dir": "/bench/runs",
    "cache_dir": "/bench/cache",
    "metrics_port": 9464,
    "log_level": "INFO"
  },
  "artifact_store": {
    "type": "local",
    "uri": "/bench/runs"
  }
}

```

Scientific versus operational fields

Some fields define the scientific identity of the run. Changing them changes the experiment:

- benchmark
- framework
- framework_config.sha256
- dataset
- selection
- models.answerer.requested_model
- models.answerer.parameters
- models.judge.requested_model
- models.judge.parameters
- evaluation

Other fields are operational. They can change where or how the same experiment is executed, but they should still be recorded for auditability:

- runtime.runs_dir
- runtime.cache_dir
- runtime.metrics_port
- runtime.log_level
- qdrant.request_timeout_seconds
- artifact_store.uri

dmf-bench records both kinds of information in the run manifest and provenance artifacts, but the scientific fingerprint focuses on the scientific inputs.

Minimal shape

A complete config has this shape:

```

{
  "schema_version": 2,
  "experiment_id": "locomo-dmf-official-run",
  "benchmark": "locomo",
  "framework": "dmf",
  "runtime": {
    "root": "/bench",
    "runs_dir": "/bench/runs",
    "cache_dir": "/bench/cache",
    "metrics_port": 9464,
    "log_level": "INFO"
  },
  "framework_config": {
    "path": "/bench/config/locomo_dmf_qdrant_settings.toml",
    "sha256": "64_character_sha256",

```

```

    "format": "toml",
    "profile": "official-v1"
  },
  "qdrant": {
    "endpoint_env": "QDRANT_URL",
    "retention": "keep",
    "request_timeout_seconds": 10
  },
  "dataset": {
    "name": "locomo",
    "registry_id": "locomo-official-unpinned",
    "allow_unpinned": true
  },
  "selection": {
    "ordered_item_ids": ["conversation-0001"],
    "filters": {},
    "seed": 7
  },
  "models": {
    "answerer": {
      "provider": "openai",
      "requested_model": "gpt-4.1-mini",
      "parameters": {
        "temperature": 0,
        "max_tokens": 4096
      },
      "runtime": {
        "timeout_seconds": 120,
        "rpm": 200,
        "max_retries": 5
      }
    },
    "judge": {
      "provider": "openai",
      "requested_model": "gpt-5-mini",
      "parameters": {
        "temperature": 0,
        "max_tokens": 4096,
        "reasoning_effort": "low"
      },
      "runtime": {
        "timeout_seconds": 120,
        "rpm": 200,
        "max_retries": 5
      }
    }
  },
  "evaluation": {
    "required": ["primary_judge_score", "rigorous_report"],
    "optional": ["ablation_report"],
    "ablation_required": false
  },
  "artifact_store": {
    "type": "local",
    "uri": "/bench/runs"
  }
}

```

```
}
}
```

Before running an experiment, validate the config:

```
dmf-bench validate --materialize-datasets --config path/to/experiment.json
```

Validation checks the config schema, path boundaries, framework config hash, dataset declaration, and environment-dependent settings that can be checked without executing the benchmark lifecycle.

5. Dataset

`dmf-bench` supports two ways to declare a dataset.

Already materialized dataset

Use this form when the dataset JSON file is already available:

```
{
  "dataset": {
    "name": "locomo",
    "source": "local-approved-dataset",
    "revision": "v1",
    "path": "/bench/datasets/locomo/locomo10.json",
    "sha256": "64_character_sha256",
    "expected_schema": "locomo10-json-array-v1"
  }
}
```

In this mode, `dmf-bench validate` checks that the file exists and that the SHA-256 matches.

Registry dataset

Use this form when you want `dmf-bench` to download or reuse a registered source:

```
{
  "dataset": {
    "name": "locomo",
    "registry_id": "locomo-official-unpinned",
    "allow_unpinned": true
  }
}
```

During `run`, the dataset is materialized under:

```
/bench/cache/datasets/<fingerprint>/
```

The directory always contains a `manifest.json`. The manifest documents:

- dataset id, benchmark, source URL, and revision;
- path and SHA-256 of the observed source file;
- path and SHA-256 of the materialized file;
- file sizes;
- optional sampling policy.

If the dataset is marked `unpinned`, `allow_unpinned: true` is required. This makes explicit that the run accepts a source that is not locked by an approved hash.

6. Sampling

Sampling is optional and is declared inside `dataset.sampling`.

LoCoMo example, 5% of QA stratified by category:

```
{
  "dataset": {
    "name": "locomo",
    "registry_id": "locomo-official-unpinned",
    "allow_unpinned": true,
    "sampling": {
      "fraction": 0.05,
      "unit": "qa",
      "stratify_by": "category",
      "seed": 7,
      "rounding": "ceil"
    }
  }
}
```

LongMemEval example, 5% of records:

```
{
  "dataset": {
    "name": "longmemeval",
    "registry_id": "longmemeval-s-official-unpinned",
    "allow_unpinned": true,
    "sampling": {
      "fraction": 0.05,
      "unit": "record",
      "seed": 7,
      "rounding": "ceil"
    }
  }
}
```

If `sampling` is omitted, `manifest.json` is still generated, but without a `sampling` block.

7. Selection

`selection.ordered_item_ids` decides which units are executed and in which order.

For LoCoMo, IDs are conversations, for example:

```
{
  "selection": {
    "ordered_item_ids": ["conversation-0001", "conversation-0002"],
    "filters": {"categories": [1, 2, 3, 4, 5]},
    "seed": 7
  }
}
```

For LongMemEval, IDs are `question_id` values, for example:

```
{
  "selection": {
    "ordered_item_ids": ["10d9b85a", "gpt4_45189cb4"],
    "filters": {},
    "seed": 7
  }
}
```

```
}
}
```

The selection must be consistent with the materialized dataset. If sampling is used, the selected IDs must exist in the generated sample.

8. Framework And Models

`framework_config` points to the framework configuration:

- DMF uses TOML files.
- Mem0 uses YAML files.

The file must provide `path`, `sha256`, `format`, and optionally `profile`.

`models.answerer` and `models.judge` define provider, model, and parameters. Example:

```
{
  "models": {
    "answerer": {
      "provider": "openai",
      "requested_model": "gpt-4.1-mini",
      "parameters": {"temperature": 0, "max_tokens": 4096},
      "runtime": {"timeout_seconds": 120, "rpm": 200, "max_retries": 5}
    },
    "judge": {
      "provider": "openai",
      "requested_model": "gpt-5-mini",
      "parameters": {
        "temperature": 0,
        "max_tokens": 4096,
        "reasoning_effort": "low"
      },
      "runtime": {"timeout_seconds": 120, "rpm": 200, "max_retries": 5}
    }
  }
}
```

Do not add `api_key`, `token`, `secret`, or similar fields to JSON files: validation rejects them.

9. Main Commands

All user commands go through the `dmf-bench` CLI. Paths declared in the config must be valid in the environment where the CLI is running. If the CLI runs inside the official image, typical paths are `/bench/config`, `/bench/cache`, and `/bench/runs`; if it runs on the host, use host-local paths.

List benchmarks and frameworks:

```
dmf-bench list
```

List registered datasets:

```
dmf-bench list-datasets
```

Validate an already materialized configuration:

```
dmf-bench validate --config config/experiment.json
```

Validate after materializing the dataset from the registry:

```
dmf-bench validate --materialize-datasets --config config/experiment.json
```

Manually materialize a dataset:

```
dmf-bench materialize-dataset \  
  --dataset-id locomo-official-unpinned \  
  --allow-unpinned \  
  --sample-fraction 0.05 \  
  --sample-unit qa \  
  --sample-stratify-by category \  
  --sample-seed 7 \  
  --sample-rounding ceil \  
  --output-dir .cache/datasets/manual-locomo-smoke
```

The command prints a `dataset` fragment usable in a configuration and the path to `manifest.json`.

10. Execution

Before launching the run, set the environment variables required by the config, for example:

```
export OPENAI_API_KEY=...  
export QDRANT_URL=http://127.0.0.1:6333
```

Launch a run:

```
dmf-bench run --config config/experiment.json --run-id RUN_ID
```

`RUN_ID` must be unique and readable, for example:

```
official-20260809-locomo-dmf
```

During the run, JSON events are printed. A completed run exits with code 0 and prints a final `COMPLETED` state.

11. Artifact API

The Artifact API exposes results in read-only mode. When started locally, it normally responds at:

```
http://127.0.0.1:8000
```

Health check:

```
curl --fail http://127.0.0.1:8000/health
```

Run state:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID
```

Artifacts:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts
```

The run state reports the final result and the local path to the `COMPLETED.json` marker:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID
```

To download a single artifact, use exactly one of the paths returned by `/runs/RUN_ID/artifacts`.

Cost and timing metrics:

```
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts/reports/usage.json  
curl --fail http://127.0.0.1:8000/runs/RUN_ID/artifacts/reports/timing.json
```


12. Verification And Resume

Verify a published run:

```
dmf-bench verify --run-id RUN_ID --runs-dir runs
```

Inspect state:

```
dmf-bench status --run-id RUN_ID --runs-dir runs
```

Resume an interrupted run:

```
dmf-bench resume --run-id RUN_ID --runs-dir runs
```

Do not relaunch a run with the same `RUN_ID` before checking its state: you may duplicate provider calls.

13. Official Results

The official run result is published in the `/bench/runs` volume.

The most important files are:

Artifact	Content
<code>final/COMPLETED.json</code>	Final state, counts, and main references.
<code>final/manifest.json</code>	Run publication manifest.
<code>run-manifest.json</code>	Scientific run manifest.
<code>resolved-config.json</code>	Resolved configuration without secrets.
<code>reports/usage.json</code>	Tokens and provider usage when available.
<code>reports/timing.json</code>	Timing for measured phases and operations.
<code>reports/rigorous_report.json</code>	Deterministic rigorous metrics.
<code>reports/ablation_report.json</code>	Optional ablation report, if requested.
<code>logs/events.jsonl</code>	Lifecycle JSON events.
<code>datasets/materialization-manifest.json</code>	Dataset manifest copied into the run when the dataset is materialized from the registry.

To archive an official run, publish the run OCI bundle or keep at least the image name and digest, `RUN_ID`, `resolved-config.json`, dataset materialization manifest, `final/COMPLETED.json`, `reports/usage.json`, and `reports/timing.json`.

14. OCI Publication

A completed run can be published to GHCR as an OCI artifact. The artifact is not an executable image: it is a complete snapshot of the run directory, meaning everything under `runs/RUN_ID/`.

Before publication, `dmf-bench` adds these files to the run directory:

File	Purpose
<code>OFFICIAL-RUN.json</code>	Bundle metadata: OCI ref, subject, image, harness commit, and run verification.
<code>SHA256SUMS</code>	SHA-256 for every run file, excluding <code>SHA256SUMS</code> itself.

Command:

```
dmf-bench publish-run-oci \
  --run-id RUN_ID \
  --runs-dir runs \
  --ref ghcr.io/ORG/dmf-benchmarks-runs:RUN_ID \
  --subject ghcr.io/ORG/dmf-benchmarks@sha256:IMAGE_DIGEST
```

--ref is the GHCR destination for the run bundle. --subject links the bundle to the Docker image that produced the run.

To test packaging without publishing:

```
dmf-bench publish-run-oci \
  --run-id RUN_ID \
  --runs-dir runs \
  --ref ghcr.io/ORG/dmf-benchmarks-runs:RUN_ID \
  --output-dir bundles \
  --dry-run
```

A real push requires oras installed and authenticated to GHCR.

15. Ready-To-Use Example

The `smoke/` directory contains ready-to-use configurations for a restricted but substantial check:

- LoCoMo with DMF: `smoke/config/experiment-locomo-dmf.json`
- LoCoMo with Mem0: `smoke/config/experiment-locomo-mem0.json`
- LongMemEval with DMF: `smoke/config/experiment-longmemeval-dmf.json`
- LongMemEval with Mem0: `smoke/config/experiment-longmemeval-mem0.json`

If the CLI can see the `smoke/` directory, you can use the ready configs directly:

```
dmf-bench run \
  --config smoke/config/experiment-locomo-dmf.json \
  --run-id smoke-20260809-locomo-dmf
```

This is only an operational example. For a real experiment, create a dedicated configuration and treat dataset, selection, models, and costs as explicit decisions.

16. Common Problems

Missing OPENAI_API_KEY. Check the CLI environment. Do not print the key in logs.

Config not found. Verify that the path used in the command exists in the environment where `dmf-bench` runs.

Dataset not found. If the config uses `registry_id`, use `run` or `validate --materialize-datasets`. If it uses `dataset.path`, check mounts and SHA-256.

SHA-256 mismatch. The file is not the one declared. Do not change the hash to make validation pass: regenerate or replace the file with the approved one.

Read-only file system. This is normal. Write only to `/bench/cache` and `/bench/runs`.

Qdrant not reachable. Check that the `qdrant` service is running and that the run uses `QDRANT_URL=http://qdrant:6333`.

Download blocked. If `HF_HUB_OFFLINE=1`, Hugging Face downloads do not start. Prepare the cache before the run or change policy only for a controlled phase.

Provider rate limit or timeout. Check `logs/events.jsonl`, `reports/usage.json`, and run state before retrying.

OCI publication failed. Check that `oras` is installed, GHCR login is valid, and `dmf-bench verify --run-id RUN_ID --runs-dir runs` passes before pushing.

17. Operator Or Agent Checklist

1. Verify that `dmf-bench --help` responds.
2. Prepare `.env` without committing secrets.
3. Prepare or choose the JSON config.
4. Run `validate`; use `--materialize-datasets` if the config uses the registry.
5. Read the dataset `manifest.json` if the dataset was materialized.
6. Start Qdrant and the Artifact API.
7. Launch exactly one run with a unique `RUN_ID`.
8. Check exit code and `COMPLETED` state.
9. Read artifacts from the Artifact API.
10. Save usage, timing, resolved config, and dataset manifest.
11. Publish the OCI bundle with `publish-run-oci` if the run is official.
12. Stop services without deleting volumes, unless explicitly requested.